

论软件架构评估方法及其应用

⚠ **关于本文中的数字** 本文的量化数据（评估轮次、改进项数量、效果提升等）均为**示例值**，用于展示论文应有的完整度与写法。**架构类数字**（表数量、接口数、配置值）已从项目源码核实，可直接使用；**业务规模类数字**（门店数、用户数、业务量、改善幅度）在项目资料中尚未确定，属示例值，背诵前请替换为你的**真实数据**。文末附有「待替换数字清单」，便于逐项核对。

试题

论软件架构评估方法及其应用

软件架构评估是指在架构设计完成后或架构演化过程中，对架构方案是否满足系统的质量属性需求进行系统化评价的过程。架构评估能够及早发现架构设计中的风险与缺陷，避免在系统实现甚至上线之后才发现问题，从而大幅降低返工成本。常用的架构评估方法包括架构权衡分析法（ATAM）、软件架构分析法（SAAM）以及成本效益分析法（CBAM）等。

请围绕“软件架构评估方法及其应用”论题，依次从以下三个方面进行论述：

- 简要叙述你参与设计和开发的系统，以及你所担任的工作；
- 详细论述你在该系统中采用的架构评估方法，包括评估的步骤与质量属性场景的构建方法；
- 分析架构评估的结论，以及你是如何根据评估结果改进架构设计的。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台为例，论述了架构权衡分析法（ATAM）在该项目中的应用。该平台服务日本 H.I.S. 集团营业本部与旅行者，承载预约管理、通知发送、消息推送与主数据联动等核心业务，对性能、可用性、安全性与可修改性均有明确的质量属性要求。我在项目中担任系统架构师，负责质量属性需求的梳理、架构评估的组织与评估结论的落地。在架构设计完成后，我组织业务方、开发团队与运维团队共同实施了一轮架构评估：先以效用树的形式逐层分解质量属性并标注优先级，再以“刺激源—刺激—环境—制品—响应—响应度量”六要素构建具体场景，最后通过场景走查识别出 12 个敏感点、4 个权衡点与 3 个风险点。针对评估发现的读写资源争抢、依赖版本漂移、认证服务单点等问题，我分别采取了读写数据源隔离、统一版本管理插件、用户信息会话缓存三项改进措施，使关键接口响应时间与构建可复现性均得到明显改善。

正文

一、项目背景与评估动因

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务 H.I.S. 集团 300 余家营业门店与约 5000 名业务人员，日均处理预约约 3 万笔、消息与通知约 12 万条。我在项目中担任系统架构师，负责质量属性需求的梳理、架构评估的组织以及评估结论的落地。

在架构方案初步成型后，我决定在投入大规模编码之前组织一轮正式的架构评估，动因有三个。

其一是**质量属性之间存在真实的冲突**。业务方希望预约查询尽可能快，运维方希望批量作业尽可能稳，安全方希望每一次访问都做完整校验。这些诉求单独看都合理，但合在一起就会互相牵制。仅靠架构师的经验取舍，很难让各方信服，也难以保证没有遗漏。

其二是**架构决策的影响往往在编码后期才显现**。例如我们最初计划让批量作业与在线业务共用数据库连接池，这个决策在编码阶段看不出问题，只有在夜间批处理峰值与在线请求并发时才暴露。评估的价值正在于把这类问题提前到设计阶段发现。

其三是**参与方众多，需要建立共识**。平台涉及业务、开发、运维、安全与集团多个团队，各方的关注点不同。评估过程本身就是一个建立共同理解的过程。

在多种评估方法中，我选择了 ATAM。理由是：SAAM 侧重于可修改性，对性能、可用性等属性覆盖不足；CBAM 侧重于经济决策，适用于方案选型阶段而非架构评价；而 ATAM 能够同时并在多个质量属性之间进行权衡分析，并且明确产出敏感点、权衡点与风险点，正好对应我们“多属性冲突”的核心诉求。

二、评估方法的设计与实施步骤

我组织的评估共分五个阶段，参与人包括业务方代表、开发骨干、运维负责人与我本人。

第一阶段：介绍评估方法与业务驱动因素。我首先向参与方说明 ATAM 的基本概念与分析流程，使各方建立共同语言；随后由业务方阐述平台的业务目标、约束条件与主要利益相关方。这一阶段产出的关键结论有三条：旅游业具有强季节性，预约高峰期的性能是业务的刚性要求；平台的可用性直接影响订单转化，不可用即意味着直接损失；平台需长期承载业务规则变化，可修改性关乎长期成本。

第二阶段：介绍架构方案。我以视图的形式向参与方介绍了架构方案，包括部署视图（Web、Service、Batch、FaaS 四类部署单元）、数据视图（关系库承载强一致交易、云数据库承载半结构化数据、对象存储承载文件实体）、通信视图（内部 gRPC、对外 REST）以及集成视图（契约单一来源，三语言产物生成）。

第三阶段：构建质量属性效用树。这是 ATAM 的核心环节，也是我投入精力最多的部分。效用树的构建方法是自上而下逐层细化：第一层是“效用”，第二层是质量属性类别，第三层是属性的细化方向，第四层是具体的质量属性场景，并为每个场景标注优先级（高、中、低）与实现难度（高、中、低）。

我们最终形成的效用树包含四个属性分支。性能分支下细化了两个方向：预约查询的响应时间、批量作业的执行时长。可用性分支下细化了三个方向：批量作业不影响在线服务、消息不重复不遗漏、单实例故障不影响全站。安全性分支下细化了三个方向：越权访问的抵御、凭证泄露的抵御、敏感数据传输的防窃听。可修改性分支下细化了三个方向：业务规则变化的支持、多语言调用方的接口变更支持、新门店上线的支持。

第四阶段：场景走查与架构分析。我逐个场景地走查架构方案，判断该场景能否被当前设计满足，并据此识别四类要素。

第五阶段：结果汇总与优先级排序。我把识别出的要素按处理的紧迫程度排序，形成改进清单并分配到责任人。

三、评估的发现与架构改进

关于质量属性场景的构建方法。这里需要重点说明，因为场景构建的质量直接决定了评估的质量。我采用“刺激源—刺激—环境—制品—响应—响应度量”六要素来构建每一个场景，缺一不可，尤其是“响应度量”——没有量化度量的场景是无法验证的。

举一个性能场景的例子：刺激源是营业本部的操作员；刺激是在营业高峰期发起预约查询请求；环境是系统正常运行且批量作业同时执行；制品是平台的预约查询接口；响应是接口返回查询结果；响应度量是响应时间不超过 2 秒。

再举一个可用性场景的例子：刺激源是消息投递网关；刺激是对同一业务消息返回超时；环境是系统正常运行；制品是平台的消息推送链路；响应是平台正确处理重试且不产生重复发送；响应度量是重复发送率为 0。

评估发现的敏感点。 敏感点是指对某个质量属性有显著影响的架构决策。本次评估共识别出 12 个，其中三个最为关键。

第一个敏感点是数据库连接池的共享方式。评估中我们通过场景走查发现，若批量作业与在线业务共享连接池，在“批量作业与在线请求并发”这一场景下，性能度量必然不达标。这个发现直接改变了我们后续的设计。

第二个敏感点是依赖版本的管理方式。评估时我们发现部分模块采用了动态版本区间，这意味着构建产物不具备可复现性——同一个提交在不同时间、不同机器上可能解析出不同的依赖版本。这一敏感点影响的是“可修改性”与“可维护性”。

第三个敏感点是用户身份信息的获取位置。平台采用集团统一认证服务进行身份认证，若每次请求都实时向认证服务校验，则认证服务的可用性就直接决定了平台的可用性。

评估发现的权衡点。 权衡点是同时影响多个质量属性、且对一方有利必然对另一方不利的敏感点。本次共识别出 4 个。

权衡点之一是数据存储的一致性策略。我们采用混合存储并以事件驱动实现跨存储同步，这一决策对性能有利（主链路响应时间从 1.8 秒降至 420 毫秒），但对一致性不利（只能达到秒级最终一致）。评估中我们明确记录了这一权衡，并确认了它的适用边界：预约写入等强一致场景仍走事务，只有报表与通知场景才走异步链路。

权衡点之二是计算形态的选择。将数据同步设计为无服务器函数，对弹性与成本有利（平峰期成本下降约 35%），但对延迟与可调试性不利（存在冷启动，且本地难以完整复现云上触发链路）。

权衡点之三是缓存策略。为降低对外部主数据平台的依赖，我们建立了本地只读副本，这对可用性有利，但对数据新鲜度不利——业务侧可能读到数小时前的数据。

权衡点之四是粒度划分。函数按业务域而非按表划分，对一致性与可维护性有利，但对扩容精细度不利——某个低频实体的变更需要整个业务域的函数一起扩缩容。

评估发现的风险点。 风险点是可能导致质量属性不达标、但尚未确定的架构决策。本次共识别出 3 个，其中最紧迫的是“外部系统故障的传导风险”——邮件网关与消息推送服务是外部依赖，若同步调用，外部超时将阻塞平台线程。

四、改进措施的落实

针对评估结论，我推动了三项重点改进。

改进一：读写数据源隔离。 针对连接池共享这一敏感点，我为批处理配置了独立的读写数据源，读取使用只读账号，写入使用写账号，并通过代码包路径隔离两套数据访问接口。同时把事务隔离级别设置为读已提交以减少锁持有时间，把批量游标限制在 200 条以防止内存溢出。改造后，批量作业执行期间在线接口的响应时间稳定在 320 毫秒以内，批量作业自身耗时反而下降约 30%。

改进二：统一依赖版本管理。 针对依赖版本的敏感点，我建立了独立的统一版本管理插件，各模块通过插件接入统一版本源，关键组件使用强制版本策略锁定。改造后，同一提交在任何机器上的构建产物完全一致。这项改进的直接触发因素，是评估后不久发生的一次真实故障——某次依赖升级后 protobuf 公共类型库被解析到候选版本，导致服务启动失败，恰好印证了评估的预判。

改进三：用户信息会话缓存。 针对认证服务单点的敏感点，我把用户身份信息在登录后缓存于服务端会话中，使认证服务短暂不可用时，已登录用户仍可继续操作。这把这个风险的影响范围从“全站不可用”降级为“新用户暂时

无法登录"。

五、总结与展望

本次架构评估共识别出 12 个敏感点、4 个权衡点与 3 个风险点，形成改进清单 10 项，其中 8 项已在架构定型前落实，2 项列为后续计划。平台上线后稳定运行约 8 个月，年可用率达到 99.95%，关键接口响应时间、构建可复现性均达到评估时设定的目标。

评估带来的最大收益不是发现了某个具体缺陷，而是**让质量属性的取舍变得显式**。在评估之前，很多决策只存在于架构师的脑中；评估之后，每一项权衡都被记录、被讨论、被确认，参与方对"为什么这样设计"有了共同的理解。这一点在后续的需求变更讨论中体现得尤为明显——当有人提出"能不能把同步改成实时"时，我们不再需要重新论证，只需查阅当时的权衡记录。

目前仍存在两点不足：一是本次评估为一次性活动，尚未形成定期复评机制，架构演化后的质量属性变化缺少持续跟踪；二是评估主要依赖人工走查，缺少量化的架构度量手段，如耦合度、圈复杂度等指标的自动采集。后续计划把架构评估纳入版本迭代流程，每季度复评一次，并引入自动化度量工具辅助评估。

通过本项目，我对架构评估形成了三点认识。第一，评估的价值在于"提前"——在设计阶段发现问题的成本，远低于在上线之后。第二，评估的产出不仅是问题清单，更是共识；让利益相关方共同参与场景构建的过程，本身就是消解分歧的过程。第三，场景的六要素中，"响应度量"最容易被忽略，但它恰恰是评估能否被验证的关键——没有度量的场景只是一句愿望。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

位置	示例值	请替换为
门店数 / 业务人员数	300 余家 / 约 5000 名	真实数量
日均预约量 / 消息量	约 3 万笔 / 约 12 万条	真实数量
敏感点 / 权衡点 / 风险点数量	12 / 4 / 3	如实填写
改进项数量	共 10 项, 落实 8 项	如实填写
主链路响应时间	1.8 秒 → 420 毫秒	真实数据
平峰期成本下降	约 35%	真实数据
改造后响应时间	320 毫秒以内	真实数据
批量作业耗时下降	约 30%	真实数据
稳定运行时长 / 可用率	约 8 个月 / 99.95%	真实指标

本文字数

摘要约 350 字，正文约 2950 字，**合计约 3300 字**（略超，考场可酌情精简第四节）。

可复用要点

- **ATAM 五阶段 + 效用树 + 六要素场景 + 四类要素（敏感点/权衡点/风险点/非风险点）** 是本题的标准骨架，必须背熟
- **场景六要素必须能默写**：刺激源、刺激、环境、制品、响应、响应度量。绝大多数考生会漏掉"响应度量"
- **把项目真实做过的取舍改写成"权衡点"**，比空谈方法论有说服力得多（本文列的 4 个权衡点全部来自项目真实决策）
- **「评估最大的收益是让取舍变得显式」** 是一个高层次的收尾判断
- 若题目指定 SAAM 或 CBAM，把第三节的方法论替换即可，项目素材完全通用